

# DB接続版 — コード全体とファイル構成

モックデータ版の `dao` パッケージを、実際にMySQLへ接続するコードに書き換えた場合の **完成イメージ**をまとめたものです（[02 DB接続ガイド.md](#) の内容を、全DAOクラス分まで実際に書き起こしたもの）。

⚠ これは「こう書けば動く」という参考コードです。実際に動かすには、別途MySQLサーバーを用意し、テーブルを作成し、JDBCドライバをプロジェクトに追加する必要があります（[02 DB接続ガイド.md](#) の「2. 接続までの全体準備」を参照）。

## 1. ファイル構成（モック版との違い）

`entity` / `model` / `control` / `jsp` は一切変更不要です。変わるのは `dao` パッケージの中身と、新規追加する接続共通クラス `DBUtil.java` だけです。

```
za-navi/
├─ src/
│   ├─ control/
│   │   └─ ControlServlet.java      ... 変更なし
│   ├─ dao/
│   │   ├─ DBUtil.java             ... ★新規追加（接続情報の共通クラス）
│   │   ├─ AccountsDAO.java        ... ★モックデータ→JDBCに書き換え
│   │   ├─ LocationsDAO.java       ... ★同上
│   │   ├─ MastersDAO.java         ... ★同上
│   │   ├─ ReservationsDAO.java    ... ★同上
│   │   └─ SeatsDAO.java           ... ★同上
│   ├─ entity/                    ... 変更なし（Account / Location / Reservation / Seat）
│   └─ model/                     ... 変更なし（各Logicクラス）
└─ WebContent/                   ... 変更なし（JSP・css・images）
```

## 2. MySQLのテーブル定義（DB側の準備）

各 `entity` クラスのフィールドをもとに、以下の4テーブルを作成しておきます。

```

CREATE TABLE accounts (
    user_id   VARCHAR(20) PRIMARY KEY,
    password  VARCHAR(50) NOT NULL,
    name      VARCHAR(50) NOT NULL,
    b_id      INT          NOT NULL,
    admin     INT          NOT NULL DEFAULT 0
);

CREATE TABLE departments (
    b_id      INT          PRIMARY KEY,
    b_name    VARCHAR(50) NOT NULL
);

CREATE TABLE seats (
    seat_id   INT AUTO_INCREMENT PRIMARY KEY,
    base_name VARCHAR(50) NOT NULL,
    f_name    VARCHAR(20) NOT NULL,
    area_name VARCHAR(50) NOT NULL,
    seat_name VARCHAR(20) NOT NULL
);

CREATE TABLE reservations (
    reserve_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id    VARCHAR(20) NOT NULL,
    register_id VARCHAR(20) NOT NULL,
    reserve_date DATE      NOT NULL,
    gyosaki    VARCHAR(10) NOT NULL,
    memo       VARCHAR(200),
    seat_id    INT          NOT NULL DEFAULT 0,
    cancel_flag INT          NOT NULL DEFAULT 0,
    cancel_date DATE
);

```

seats と reservations は AUTO\_INCREMENT にしておくと、MastersDAO#add / ReservationsDAO#add で発行されたIDを getGeneratedKeys() から取得できます。

### 3. DBUtil.java (★新規追加：接続共通クラス)

---

```
package dao;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

/**
 * MySQL接続用の共通クラス。
 * 全DAOがここを通して Connection を取得する（接続情報を1箇所にまとめておくため）。
 */
public class DBUtil {
    private static final String URL = "jdbc:mysql://localhost:3306/za_navi_db?useSSL=false&serverTimezone=Asia/Tokyo&characterEncoding=UTF-8";
    private static final String USER = "root";
    private static final String PASS = "your_password";

    static {
        try {
            // ドライバを明示的に読み込んでDriverManagerに登録する
            Class.forName("com.mysql.cj.jdbc.Driver");
        } catch (ClassNotFoundException e) {
            throw new RuntimeException("MySQL JDBCドライバが見つかりません。jarファイルを追加したか確認してください。", e);
        }
    }

    public static Connection getConnection() throws SQLException {
        // 登録済みのドライバの中から、URLの "jdbc:mysql://" に対応するものをDriverManagerが選んで接続する
        return DriverManager.getConnection(URL, USER, PASS);
    }
}
```

---

## 4. AccountsDAO.java (DB接続版)

---

```
package dao;

import entity.Account;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.List;

public class AccountsDAO {

    public Account findById(String userId) {
        String sql = "SELECT * FROM accounts WHERE user_id = ?";
        try (Connection con = DBUtil.getConnection();
            PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setString(1, userId);
            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next()) return toAccount(rs);
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
        return null;
    }

    public Account authenticate(String userId, String password) {
        String sql = "SELECT * FROM accounts WHERE user_id = ? AND password = ?";
        try (Connection con = DBUtil.getConnection();
            PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setString(1, userId);
            ps.setString(2, password);
            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next()) return toAccount(rs);
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
        return null;
    }
}
```

```
}
```

```
public List<Account> findAll() {  
    List<Account> list = new ArrayList<>();  
    String sql = "SELECT * FROM accounts ORDER BY user_id";  
    try (Connection con = DBUtil.getConnection();  
        PreparedStatement ps = con.prepareStatement(sql);  
        ResultSet rs = ps.executeQuery()) {  
        while (rs.next()) list.add(toAccount(rs));  
    } catch (SQLException e) {  
        e.printStackTrace();  
    }  
    return list;  
}
```

```
public List<Account> findByName(String keyword) {  
    List<Account> list = new ArrayList<>();  
    String sql = "SELECT * FROM accounts WHERE name LIKE ? ORDER BY user_id";  
    try (Connection con = DBUtil.getConnection();  
        PreparedStatement ps = con.prepareStatement(sql)) {  
        ps.setString(1, "%" + (keyword == null ? "" : keyword) + "%");  
        try (ResultSet rs = ps.executeQuery()) {  
            while (rs.next()) list.add(toAccount(rs));  
        }  
    } catch (SQLException e) {  
        e.printStackTrace();  
    }  
    return list;  
}
```

```
public List<Account> findByBId(int bId) {  
    List<Account> list = new ArrayList<>();  
    String sql = "SELECT * FROM accounts WHERE b_id = ? ORDER BY user_id";  
    try (Connection con = DBUtil.getConnection();  
        PreparedStatement ps = con.prepareStatement(sql)) {  
        ps.setInt(1, bId);  
        try (ResultSet rs = ps.executeQuery()) {  
            while (rs.next()) list.add(toAccount(rs));  
        }  
    } catch (SQLException e) {  
        e.printStackTrace();  
    }  
}
```

```
        return list;
    }

    /** ResultSetの1行をAccountに詰め替える共通処理 */
    private Account toAccount(ResultSet rs) throws SQLException {
        return new Account(
            rs.getString("user_id"),
            rs.getString("password"),
            rs.getString("name"),
            rs.getInt("b_id"),
            rs.getInt("admin")
        );
    }
}
```

---

## 5. LocationsDAO.java (DB接続版)

---

```
package dao;

import entity.Location;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.List;

public class LocationsDAO {

    public List<Location> findAll() {
        List<Location> list = new ArrayList<>();
        String sql = "SELECT * FROM departments ORDER BY b_id";
        try (Connection con = DBUtil.getConnection();
            PreparedStatement ps = con.prepareStatement(sql);
            ResultSet rs = ps.executeQuery()) {
            while (rs.next()) {
                list.add(new Location(rs.getInt("b_id"), rs.getString("b_name")));
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
        return list;
    }

    public Location findById(int bId) {
        String sql = "SELECT * FROM departments WHERE b_id = ?";
        try (Connection con = DBUtil.getConnection();
            PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setInt(1, bId);
            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next()) return new Location(rs.getInt("b_id"), rs.getString("b_name"));
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
        return null;
    }
}
```

```
}

public String getBNameById(int bId) {
    Location loc = findById(bId);
    return loc != null ? loc.getBName() : "不明";
}
}
```

---



## 6. SeatsDAO.java (DB接続版)

```
package dao;

import entity.Seat;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.stream.Collectors;

public class SeatsDAO {

    /**
     * 拠点（オフィス）名 → フロアマップ画像ファイル名の対応表。
     * 画像ファイル名は「マスタデータ」というよりアプリ側の固定情報なので、
     * DBには持たせずこのままアプリ内に残す。
     */
    static final Map<String, String> OFFICE_IMAGES = new LinkedHashMap<>();
    static {
        OFFICE_IMAGES.put("三田オフィス", "office_mita.png");
        OFFICE_IMAGES.put("芝浦オフィス", "office_shibaura.png");
    }

    public String getOfficeImage(String baseName) {
        return OFFICE_IMAGES.get(baseName);
    }

    public List<Seat> getAllSeats() {
        List<Seat> list = new ArrayList<>();
        String sql = "SELECT * FROM seats ORDER BY seat_id";
        try (Connection con = DBUtil.getConnection();
            PreparedStatement ps = con.prepareStatement(sql);
            ResultSet rs = ps.executeQuery()) {
            while (rs.next()) list.add(toSeat(rs));
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

```
    }  
    return list;  
}
```

```
public Seat findById(int seatId) {  
    String sql = "SELECT * FROM seats WHERE seat_id = ?";  
    try (Connection con = DBUtil.getConnection();  
        PreparedStatement ps = con.prepareStatement(sql)) {  
        ps.setInt(1, seatId);  
        try (ResultSet rs = ps.executeQuery()) {  
            if (rs.next()) return toSeat(rs);  
        }  
    } catch (SQLException e) {  
        e.printStackTrace();  
    }  
    return null;  
}
```

```
public List<Seat> findByBase(String baseName) {  
    return filterSeats(baseName, null, null);  
}
```

```
public List<Seat> findByFloor(String baseName, String fName) {  
    return filterSeats(baseName, fName, null);  
}
```

```
public List<Seat> findByArea(String baseName, String fName, String areaName) {  
    return filterSeats(baseName, fName, areaName);  
}
```

```
/** 絞り込み検索 (nullや空文字は条件なし扱い) */
```

```
public List<Seat> filterSeats(String baseName, String fName, String areaName) {  
    StringBuilder sql = new StringBuilder("SELECT * FROM seats WHERE 1=1");  
    List<String> params = new ArrayList<>();  
    if (baseName != null && !baseName.isEmpty()) { sql.append(" AND base_name = ?"); params.add(baseName);  
    }  
  
    if (fName != null && !fName.isEmpty()) { sql.append(" AND f_name = ?"); params.add(fName); }  
    if (areaName != null && !areaName.isEmpty()) { sql.append(" AND area_name = ?"); params.add(areaName);  
    }  
  
    sql.append(" ORDER BY seat_id");  
  
    List<Seat> list = new ArrayList<>();
```

```

try (Connection con = DBUtil.getConnection();
    PreparedStatement ps = con.prepareStatement(sql.toString())) {
    for (int i = 0; i < params.size(); i++) ps.setString(i + 1, params.get(i));
    try (ResultSet rs = ps.executeQuery()) {
        while (rs.next()) list.add(toSeat(rs));
    }
} catch (SQLException e) {
    e.printStackTrace();
}
return list;
}

/** 拠点一覧（重複除去） */
public List<String> getBases() {
    return getAllSeats().stream().map(Seat::getBaseName).distinct().collect(Collectors.toList());
}

/** 指定拠点のフロア一覧 */
public List<String> getFloors(String baseName) {
    return getAllSeats().stream()
        .filter(s -> baseName == null || baseName.isEmpty() || s.getBaseName().equals(baseName))
        .map(Seat::getFName).distinct().collect(Collectors.toList());
}

/** 指定拠点・フロアのエリア一覧 */
public List<String> getAreas(String baseName, String fName) {
    return getAllSeats().stream()
        .filter(s -> (baseName == null || baseName.isEmpty() || s.getBaseName().equals(baseName)))
        .filter(s -> (fName == null || fName.isEmpty() || s.getFName().equals(fName)))
        .map(Seat::getAreaName).distinct().collect(Collectors.toList());
}

private Seat toSeat(ResultSet rs) throws SQLException {
    return new Seat(
        rs.getInt("seat_id"),
        rs.getString("base_name"),
        rs.getString("f_name"),
        rs.getString("area_name"),
        rs.getString("seat_name")
    );
}

```

```
}
```

```
}
```

---

## 7. MastersDAO.java (DB接続版)

---

```
package dao;

import entity.Seat;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
import java.util.ArrayList;
import java.util.List;

public class MastersDAO {

    public List<Seat> getAll() {
        List<Seat> list = new ArrayList<>();
        String sql = "SELECT * FROM seats ORDER BY base_name, f_name, area_name, seat_name";
        try (Connection con = DBUtil.getConnection();
            PreparedStatement ps = con.prepareStatement(sql);
            ResultSet rs = ps.executeQuery()) {
            while (rs.next()) list.add(toSeat(rs));
        } catch (SQLException e) {
            e.printStackTrace();
        }
        return list;
    }

    public Seat findById(int seatId) {
        String sql = "SELECT * FROM seats WHERE seat_id = ?";
        try (Connection con = DBUtil.getConnection();
            PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setInt(1, seatId);
            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next()) return toSeat(rs);
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
        return null;
    }
}
```

```

public void add(Seat seat) {
    String sql = "INSERT INTO seats (base_name, f_name, area_name, seat_name) VALUES (?, ?, ?, ?)";
    try (Connection con = DBUtil.getConnection();
        PreparedStatement ps = con.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS)) {
        ps.setString(1, seat.getBaseName());
        ps.setString(2, seat.getFName());
        ps.setString(3, seat.getAreaName());
        ps.setString(4, seat.getSeatName());
        ps.executeUpdate();
        try (ResultSet keys = ps.getGeneratedKeys()) {
            if (keys.next()) seat.setSeatId(keys.getInt(1));
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
}

public boolean update(Seat updated) {
    String sql = "UPDATE seats SET base_name=?, f_name=?, area_name=?, seat_name=? WHERE seat_id=?";
    try (Connection con = DBUtil.getConnection();
        PreparedStatement ps = con.prepareStatement(sql)) {
        ps.setString(1, updated.getBaseName());
        ps.setString(2, updated.getFName());
        ps.setString(3, updated.getAreaName());
        ps.setString(4, updated.getSeatName());
        ps.setInt(5, updated.getSeatId());
        return ps.executeUpdate() > 0;
    } catch (SQLException e) {
        e.printStackTrace();
    }
    return false;
}

public boolean delete(int seatId) {
    String sql = "DELETE FROM seats WHERE seat_id = ?";
    try (Connection con = DBUtil.getConnection();
        PreparedStatement ps = con.prepareStatement(sql)) {
        ps.setInt(1, seatId);
        return ps.executeUpdate() > 0;
    } catch (SQLException e) {
        e.printStackTrace();
    }
}

```

```
    }  
    return false;  
}  
  
private Seat toSeat(ResultSet rs) throws SQLException {  
    return new Seat(  
        rs.getInt("seat_id"),  
        rs.getString("base_name"),  
        rs.getString("f_name"),  
        rs.getString("area_name"),  
        rs.getString("seat_name")  
    );  
}  
}
```

`seats.seat_id` を `AUTO_INCREMENT` にしてあるので、`add()` では `Statement.RETURN_GENERATED_KEYS` でDBが発行した新しいIDを取得し、`seat.setSeatId(...)` に反映しています。

---

## 8. ReservationsDAO.java (DB接続版)

---

```
package dao;

import entity.Reservation;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

public class ReservationsDAO {

    public List<Reservation> findByUserId(String userId) {
        List<Reservation> list = new ArrayList<>();
        String sql = "SELECT * FROM reservations WHERE user_id = ? AND cancel_flag = 0 ORDER BY reserve_date";
        try (Connection con = DBUtil.getConnection();
            PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setString(1, userId);
            try (ResultSet rs = ps.executeQuery()) {
                while (rs.next()) list.add(toReservation(rs));
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
        return list;
    }

    public List<Reservation> findTodayByUserId(String userId) {
        List<Reservation> list = new ArrayList<>();
        String sql = "SELECT * FROM reservations WHERE user_id = ? AND reserve_date = CURDATE() AND cancel_flag = 0";
        try (Connection con = DBUtil.getConnection();
            PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setString(1, userId);
            try (ResultSet rs = ps.executeQuery()) {
                while (rs.next()) list.add(toReservation(rs));
            }
        }
    }
}
```



```

    }
    } catch (SQLException e) {
        e.printStackTrace();
    }
    return list;
}

```

```

public List<Reservation> findByDate(String date) {
    List<Reservation> list = new ArrayList<>();
    String sql = "SELECT * FROM reservations WHERE reserve_date = ? AND cancel_flag = 0";
    try (Connection con = DBUtil.getConnection();
        PreparedStatement ps = con.prepareStatement(sql)) {
        ps.setString(1, date);
        try (ResultSet rs = ps.executeQuery()) {
            while (rs.next()) list.add(toReservation(rs));
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
    return list;
}

```

```

/**
 * 指定日のseatId→userId マップを返す（座席ステータス表示用）
 * key=seatId, value=userId
 */

```

```

public Map<Integer, String> getSeatUsageForDate(String date) {
    Map<Integer, String> map = new HashMap<>();
    String sql = "SELECT seat_id, user_id FROM reservations WHERE reserve_date = ? AND cancel_flag = 0 AND seat_id > 0";
    try (Connection con = DBUtil.getConnection();
        PreparedStatement ps = con.prepareStatement(sql)) {
        ps.setString(1, date);
        try (ResultSet rs = ps.executeQuery()) {
            while (rs.next()) map.put(rs.getInt("seat_id"), rs.getString("user_id"));
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
    return map;
}

```

```

public void add(Reservation r) {
    String sql = "INSERT INTO reservations (user_id, register_id, reserve_date, gyosaki, memo, seat_id, cancel_flag) "
        + "VALUES (?, ?, ?, ?, ?, ?, 0)";
    try (Connection con = DBUtil.getConnection();
        PreparedStatement ps = con.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS)) {
        ps.setString(1, r.getUserId());
        ps.setString(2, r.getRegisterId());
        ps.setString(3, r.getReserveDate());
        ps.setString(4, r.getGyosaki());
        ps.setString(5, r.getMemo());
        ps.setInt(6, r.getSeatId());
        ps.executeUpdate();
        try (ResultSet keys = ps.getGeneratedKeys()) {
            if (keys.next()) r.setReserveId(keys.getInt(1));
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
}

public void cancel(int reserveId) {
    String sql = "UPDATE reservations SET cancel_flag = 1, cancel_date = CURDATE() WHERE reserve_id = ?";
    try (Connection con = DBUtil.getConnection();
        PreparedStatement ps = con.prepareStatement(sql)) {
        ps.setInt(1, reserveId);
        ps.executeUpdate();
    } catch (SQLException e) {
        e.printStackTrace();
    }
}

public Reservation findById(int reserveId) {
    String sql = "SELECT * FROM reservations WHERE reserve_id = ?";
    try (Connection con = DBUtil.getConnection();
        PreparedStatement ps = con.prepareStatement(sql)) {
        ps.setInt(1, reserveId);
        try (ResultSet rs = ps.executeQuery()) {
            if (rs.next()) return toReservation(rs);
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
}

```

```

    }

    return null;
}

private Reservation toReservation(ResultSet rs) throws SQLException {
    return new Reservation(
        rs.getInt("reserve_id"),
        rs.getString("user_id"),
        rs.getString("register_id"),
        rs.getString("reserve_date"),
        rs.getString("gyosaki"),
        rs.getString("memo"),
        rs.getInt("seat_id"),
        rs.getInt("cancel_flag"),
        rs.getString("cancel_date")
    );
}
}

```

## 9. 書き換えのポイント整理

- どのDAOも、メソッドの**名前・引数・戻り値の型はモック版と完全に同じ**にしています。そのため `model` (Logic) や `JSP` 側は一切修正不要です。
- `try (Connection con = ...; PreparedStatement ps = ...)` の形 (try-with-resources) を使うと、`Connection` や `PreparedStatement` を閉じ忘れる心配がありません。
- `SELECT` 系は「SQLを書く → パラメータをセット → 実行 → ResultSetから1行ずつentityに詰め替える」、`INSERT/UPDATE/DELETE` 系は「SQLを書く → パラメータをセット → `executeUpdate()`」という、同じ「型」の繰り返しになっています。1つ書き方を覚えれば、あとは応用です。
- `SeatsDAO` の `OFFICE_IMAGES` (拠点名→画像ファイル名の対応表) は、DBのデータというより アプリ側の固定情報なので、DB接続後もこのままJavaのコードに残します。